Project 4-1
Implementation of a Code Generator I
Due on Thursday, December 26, 2024

1. **Problem:**

In this project you are requested to implement a code generator of **Cactus** for MIPS processors. Your code generator should generate code only if the program is free of lexical and syntactical errors. You should use the SPIM simulator, which simulates a MIPS processor, to verify the correctness of your code generator. You can download the SPIM simulator and related information from the web site http://www.cs.wisc.edu/~larus/spim.html. You can apply the techniques for intermediate code generation in this project. You should finish the code generation for variable declarations, assignments, and system calls in this project.

In this project, since there is only one procedure, you may assume that all variables are global variables. We will use a word to store an integer data. To simplify register allocation, you can use an integer counter with an initial value of zero to maintain 10 general registers consisting of $t0~$t9 for expression evaluation. You may assume that 10 registers are enough to evaluate all statements in the program. You should return all the registers you use back after evaluating a statement.

In this project, since SPIM starts the execution from the label named "main", you can always use "main" as the name of the procedure. The read, write, and exit statements should be implemented using the system calls read_int, print_int, and exit of the SPIM simulator. The code generator reads input from stdin, writes output to stdout, and writes errors to stderr.

Consider the following **Cactus** program:

```
// A simple program
    Program
    Begin
        Var  n
        Var  s
```

```
    Set s = 3
    Read n
    Set s = s + n * 2
    Write s
End
```

The generated MIPS assembly code should be as follows:

```
        .data
n:      .word  0                # Var n
s:      .word  0                # Var s
        .text
main:
        li      $t0, 3          # Set s = 3
        la      $t1, s
        sw      $t0, 0($t1)
        li      $v0, 5          # Read n
        syscall
        la      $t0, n
        sw      $v0, 0($t0)
        la      $t0, s          # Set s = s + n * 2
        lw      $t0, 0($t0)
        la      $t1, n
        lw      $t1, 0($t1)
        li      $t2, 2
        mul     $t1, $t1, $t2
        add     $t0, $t0, $t1
        la      $t1, s
        sw      $t0, 0($t1)
        la      $t0, s          # Write s
        lw      $t0, 0($t0)
        move    $a0, $t0
        li      $v0, 1
        syscall
```

2. Handing in your program

The code generator should be implemented by the C file codegen.c. The executable file should be named “cactus.exe” namely,

```
flex cactus.l
bison -d cactus.y
gcc -o cactus.exe lex.yy.c lotus.tab.c tree.c codegen.c
type input.txt | .\cactus.exe
```

To turn in the project, upload a flex file cactus.l, a bison file cactus.y, and a C file codegen.c to eCourse2 site.

3. Grading

The grading is based on the correctness of your program. The correctness will be tested by a number of test cases.

It is best to incrementally build your program so that you always have a partially-correct working program. It is also best to construct a shell script to systematically test your program.